UNIVERSIDAD TECNOLOGICA NACIONAL

Tecnicatura Universitaria en Programación

# Dog Walk App


**Estudiantes**

Gastón Lell

Gabriel Lovera

Juan Cruz Leal


11 de noviembre de 2026

# Índice


| Sección | Página |
|---|---|
| Resumen ejecutivo | 3 |
| 1. Alcance y requisitos | 4 |
| 2. Arquitectura y módulos | 5 |
| 3. Modelo de datos | 6 |
| Relaciones entre tablas | 7 |
| 4. Reglas de negocio y seguridad | 8 |
| 5. Plan de pruebas y resultados | 9 |
| 6. Cobertura y publicación | 10 |
| 7. Mascotas y búsqueda de paseos | 11 |
| 8. Reserva y cancelación | 12 |
| 9. Finalización y reseña de un paseo | 13 |
| 10. Instalación y recorrido de la demo | 14 |
| 11. Equipo, alcance y conclusiones | 15 |
| 12. Conclusiones | 16 |
| Anexo: API y referencias | 17 |
| Anexo: capturas adicionales | 18 |

# Resumen ejecutivo

Dog Walk App, también llamada Paseadores, es una aplicación web para organizar paseos grupales de perros en Buenos Aires. Los paseadores publican el barrio, el horario, el punto de encuentro, el precio y el cupo. Los dueños cargan sus mascotas y reservan un lugar por vez.

Los paseadores definen sus zonas de servicio, publican paseos y consultan su agenda. Los dueños pueden cargar una mascota al registrarse, buscar por barrio, fecha, horario y reservar para una de sus mascotas. Mis reservas es un modulo que separa los paseos próximos, en curso y completados. Un dueño puede reseñar un paseo una vez finalizado desde Historial de paseos completados

## El problema que quisimos resolver

Cuando un paseo se coordina por mensajes, el horario, el punto de encuentro y los cambios de último momento pueden quedar repartidos en distintas conversaciones. También es difícil saber cuántas mascotas reservaron. Por lo cual reunimos esos datos en la página del paseo y les dimos a ambos roles, tanto el paseado o como el dueño de mascotas un lugar para consultar los paseos y llevar un registro.

## Objetivo del proyecto

Construir una aplicación web respaldada por una base de datos relacional donde los paseadores puedan organizar paseos grupales por barrio y los dueños puedan reservar solo cuando haya lugar.

## Objetivos específicos

• Permitir que los paseadores publiquen en los barrios que cubren.
• Permitir que los dueños busquen por barrio y reserven para sus propias mascotas.
• Mantener un historial y validar permisos para cancelaciones, finalización y reseñas.
• Verificar con dos solicitudes concurrentes que solo una pueda reservar el último lugar.

Probamos ambos roles con cuentas de ejemplo y una base de datos local y luego desplegamos. A continuación se incluyen las capturas y los resultados. Agregaremos los enlaces a la aplicación pública y el video de presentación.

# 1. Alcance y requisitos

La aplicación tiene dos roles, uno es el dueño de mascota y  otro es el paseador. Cada uno tiene su propia navegación. El catálogo de barrios es público, pero los datos de la cuenta, los paseos y las reservas requieren iniciar sesión.

| Requisito | Comportamiento esperado |
|---|---|
| RF01. Cuentas y perfiles | Registrarse como dueño o paseador, iniciar sesión y editar los datos propios. Los demás usuarios no pueden ver el correo electrónico ni el teléfono del paseador. |
| RF02. Cobertura del servicio | Guardar barrios y valores sugeridos de precio y cupo. La publicación usa esos barrios de servicio. |
| RF03. Publicación | Crear un paseo futuro con duración, precio, cupo y punto de encuentro. Confirmar la publicación y mostrarla en la agenda. Rechazar paseos superpuestos. |
| RF04. Mascotas y búsqueda | Cargar y editar las mascotas propias. Buscar paseos futuros por barrio, fecha u horario y ver el detalle. |
| RF05. Reserva | Reservar para una de las mascotas propias si queda lugar y no llegó la hora de inicio. Ver Próximas, En curso e Historial. |
| RF06. Cancelación y finalización | Liberar el lugar cuando una reserva se cancela a tiempo. El paseador puede iniciar un paseo con mascotas reservadas y finalizar un paseo en curso. |
| RF07. Reseñas | Aceptar una reseña por dueño participante y paseo finalizado. Mostrar la reseña y actualizar la calificación del paseador. |

## Criterios de calidad

El servidor valida permisos y disponibilidad, y las restricciones de la base de datos evitan duplicados. También probamos la responsividad de la página. La guía de instalación explica cómo repetir el recorrido con una base de datos nueva.

## Límites del alcance

Los precios se muestran en pesos argentinos. El mapa muestra la ubicación aproximada de los barrios a través de una librería integrada y el punto de encuentro se indica como texto. La aplicación no incluye pagos en línea, seguimiento de ubicación en tiempo real, chat ni una app móvil nativa, pero son consideraciones que quedan para próximas iteraciones.

# 2. Arquitectura y módulos

La interfaz usa React, TypeScript y Vite. Spring Boot gestiona las reglas de la API y PostgreSQL almacena los datos. Flyway ejecuta las migraciones. En desarrollo, Vite redirige /api al backend. El Dockerfile de producción compila el frontend y lo sirve con Spring Boot.

| Capa | Responsabilidad |
|---|---|
| Interfaz | Pantallas de inicio de sesión, perfiles, mascotas, búsqueda, detalle del paseo, agendas, reservas y reseñas. Muestra resultados y mensajes de validación. |
| Controladores | Leen las solicitudes HTTP, identifican al usuario autenticado y convierten los resultados o errores de los servicios en respuestas de la API. |
| Servicios | Validan reglas como cobertura, superposición de horarios, titularidad de la mascota y cupo antes de permitir una operación. |
| Persistencia | Los repositorios leen y escriben registros en PostgreSQL. Las restricciones y transacciones aseguran reglas que la interfaz no puede garantizar. |

## Estructura del repositorio

| Directorio | Índice |
|---|---|
| **frontend/** | Pantallas, rutas, formularios y cliente HTTP. |
| **backend/** | Controladores, servicios, entidades, repositorios, migraciones y pruebas en Java. |
| **database/** | DDL, datos iniciales, consultas de verificación y esquema DBML editable. |
| **docs/** | Propuesta, decisiones de diseño, guía de instalación, contrato de la API y evidencia de pruebas. |

## Decisión de implementación

El módulo de paseos abarca la publicación, la búsqueda y la agenda. BookingService coordina reservas y cancelaciones. WalkPetService valida las reglas de cada reserva. Después de publicar un paseo, el paseador ve una confirmación y un enlace para compartir por Whatsapp. Su agenda separa los próximos paseos del historial.

# 3. Modelo de datos

Usamos una base de datos relacional. Las tablas siguientes muestran las relaciones principales. Las migraciones de Flyway definen las claves y los tipos exactos, y el repositorio incluye un modelo DBML editable.

| Tabla | Relaciones y campos principales |
|---|---|
| **zones** | Catálogo de barrios. Un barrio puede estar vinculado a usuarios, paseos y zonas de servicio. |
| **users** | Dueño (CLIENT) o paseador (WALKER), barrio de residencia y datos de perfil. Los paseadores pueden tener precios y cupos predeterminados y datos de calificación. |
| **walker_zones** | Vincula a un paseador con cada barrio donde ofrece paseos. |
| **pets** | Cada mascota tiene un dueño, un nombre y, opcionalmente, tamaño, raza y notas. |
| **walks** | Cada paseo pertenece a un paseador y a un barrio. Guarda horario, duración, punto de encuentro, precio, cupo y estado. |
| **walk_pets** | Vincula una mascota con un paseo. El estado de la reserva conserva el historial de cancelaciones y permite reactivar una reserva. |
| **reviews** | Una reseña pertenece a un paseo y a un dueño. Guarda una calificación, un comentario y la fecha. |

## Integridad de los datos

Los correos electrónicos son únicos sin distinguir mayúsculas y minúsculas. También son únicos los pares paseador y barrio, paseo y mascota;  paseo y autor de la reseña. Las claves foráneas exigen que existan los registros referenciados. Los controles exigen cupos y duraciones positivos, precios no negativos y calificaciones de uno a cinco. Estas validacione las aplicamos usando anotaciones correspondientes.

## Datos iniciales

La primera migración crea el esquema y la segunda carga los barrios. Empezamos la demo con una base de datos vacía y creamos las cuentas, mascotas y paseos desde la aplicación. Los precios se guardan como decimales y los horarios usan la hora local de Buenos Aires.

Referencia de diseño: modelo de datos y diagrama del repositorio.

# Relaciones entre tablas

Cada flecha va desde una clave primaria hasta el campo que la referencia. El diagrama muestra las claves y los campos más usados. La primera migración contiene las definiciones completas.

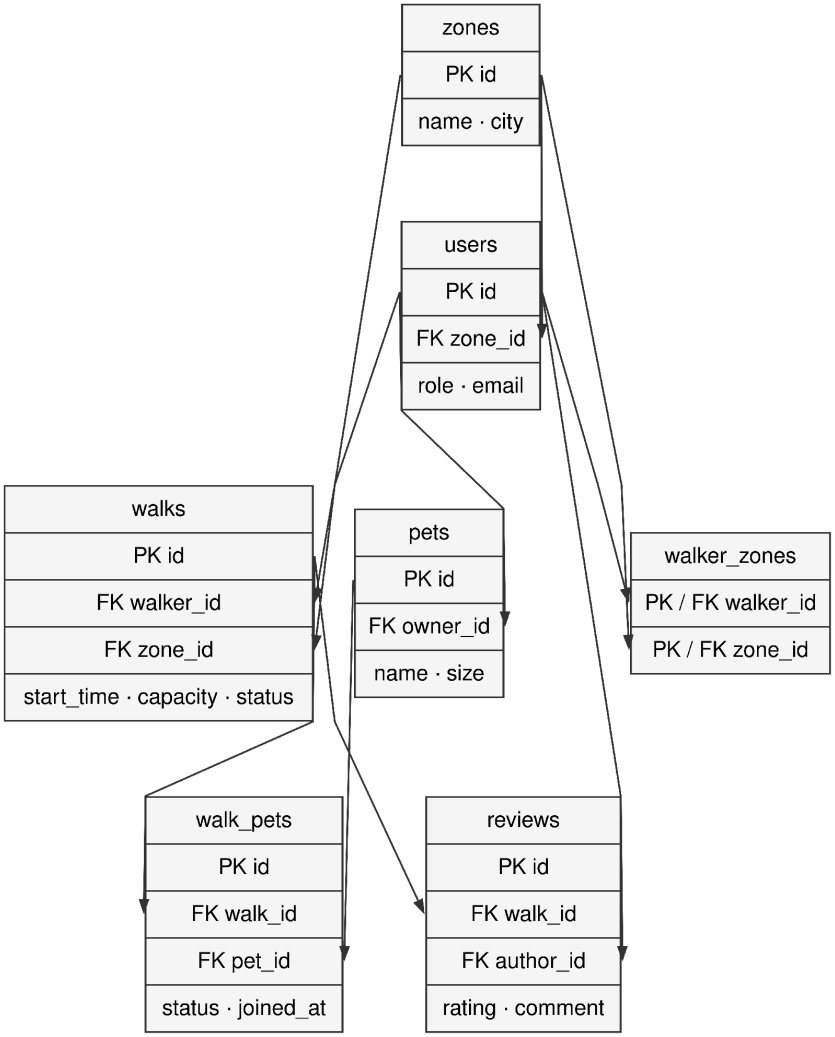



Diagrama simplificado basado en el esquema SQL. PK: clave primaria. FK: clave foránea.

## Cómo usamos estas relaciones

Cada usuario tiene un barrio de residencia. Además, un paseador puede cubrir varios barrios de servicio. Cada paseo pertenece al paseador que lo publicó y al barrio donde se realiza.

Una mascota puede participar en varios paseos y un paseo puede tener varias mascotas. La tabla walk_pets registra esas reservas y su estado. Un dueño puede escribir una reseña por paseo, aunque haya reservado para más de una mascota.

# 4. Reglas de negocio y seguridad

## Publicación y agenda

Un paseador puede publicar en un barrio que cubre, con al menos dos horas de anticipación. Un paseo debe durar como mínimo 30 minutos, en intervalos de 15 minutos. El servicio toma un bloqueo para ese paseador antes de buscar paseos superpuestos.

## Reserva y cancelación

Cada reserva corresponde a una de las mascotas del dueño. El backend bloquea la fila del paseo y luego verifica que siga publicado, que no haya llegado la hora de inicio y que haya lugar. Si dos solicitudes compiten por el último lugar, la segunda espera y vuelve a verificar la disponibilidad.

Un dueño puede cancelar una reserva programada cuando faltan más de dos horas. La cancelación queda en el historial y libera un lugar. Volver a reservar para la misma mascota reactiva el registro. Cancelar un paseo también cancela sus reservas activas.

## Estados del paseo y reseñas

| Estado | Cuándo aplica |
|---|---|
| Publicado o completo | Un paseo programado. El estado cambia automáticamente cuando se ocupa el último lugar o se libera uno. |
| En curso | El paseador puede iniciar un paseo publicado o completo si hay al menos una mascota reservada. |
| Finalizado | El paseador puede finalizar un paseo en curso. Un dueño que participó puede dejar una reseña. |
| Cancelado | El paseador puede cancelar un paseo publicado, completo o en curso. Las reservas activas se cancelan y se conserva el historial. |

Solo un dueño que participó puede reseñar un paseo finalizado, y puede hacerlo una sola vez. Al guardar la reseña se actualizan la calificación promedio y la cantidad de reseñas del paseador.

## Control de acceso

La API identifica a los usuarios autenticados con JWT y cifra las contraseñas con BCrypt. La búsqueda de paseos, el detalle del paseo y los perfiles de otros paseadores requieren iniciar sesión. Los perfiles de paseador omiten los datos de contacto privados. El servidor valida el rol y la titularidad en las operaciones de escritura. Las credenciales de la base de datos y la clave JWT se toman de variables de entorno.

# 5. Plan de pruebas y resultados

Probamos con Java 21, PostgreSQL 17 y Node.js 24. Usamos una cuenta de paseador y una de dueño, y revisamos las pantallas del dueño a 390 px.

| Verificación | Resultado |
|---|---|
| Backend | Pasaron 211 pruebas con Java 21 y PostgreSQL 17, incluidas las de integración. |
| Frontend | Pasaron 51 pruebas. El build de producción y el lint también pasaron. |
| Paseos superpuestos | Dos intentos concurrentes de publicar paseos superpuestos no pudieron concretarse ambos. |
| Último lugar | Solo una de dos solicitudes concurrentes pudo reservar el último lugar. |
| Permisos y estados | Verificamos la cancelación, la nueva reserva, los cambios de estado y el rechazo al llegar la hora de inicio. |
| Navegador y móvil | Probamos ambos roles y revisamos la búsqueda, las reservas y las reseñas a 390 px. |

## Capturas y cuentas de prueba

Con cuentas de ejemplo publicamos un paseo, reservamos el último lugar, cancelamos y volvimos a reservar. Vimos cómo la reserva pasó de Próximas a En curso y luego a Historial, donde dejamos una reseña.

## Comportamiento de las reservas

Cuando llega la hora de inicio, el paseo deja de aparecer en la búsqueda y se rechazan nuevas reservas. En Mis reservas, En curso refleja el cambio de estado que hace el paseador. Los paseos finalizados y las reservas canceladas aparecen en Historial.

# 6. Cobertura y publicación

Configuramos Palermo como barrio de servicio del paseador, con un lugar y un precio de referencia. Publicamos un paseo y revisamos la agenda. Al intentar publicar otro paseo en un horario superpuesto, la API lo rechazó.

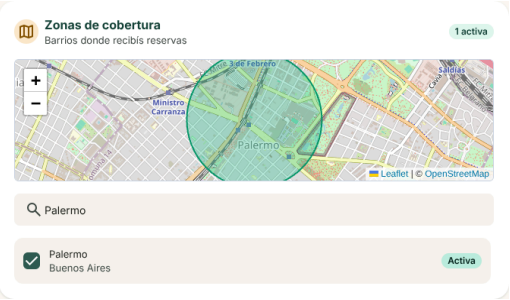



Figura 1. Palermo en la zona de servicio del paseador.

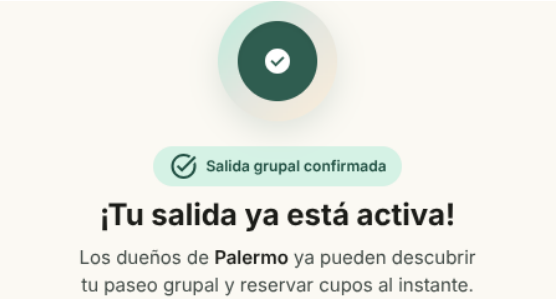



Figura 2. Confirmación después de publicar el paseo.

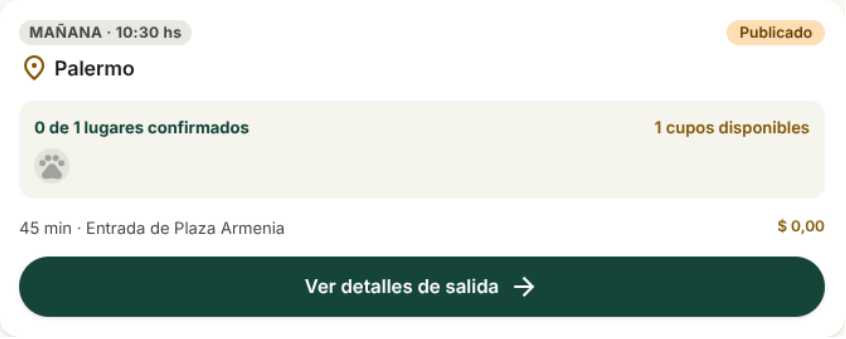



Figura 3. El paseo publicado en la agenda del paseador, con horario, cupo y punto de encuentro.

# 7. Mascotas y búsqueda de paseos

Registramos a Sofía y cargamos a Luna en el segundo paso. Buscamos por barrio y fecha. El resultado mostró el horario, el punto de encuentro y el lugar disponible.




Figura 4. Luna aparece en Mis mascotas.




Figura 5. Un resultado en Palermo con su horario, precio y un lugar disponible.

# 8. Reserva y cancelación

Reservamos el único lugar para Luna. Cancelamos desde Próximas, encontramos la cancelación en Historial y comprobamos que el lugar volvió a estar disponible. Después volvimos a reservar para la misma mascota.




Figura 6. El detalle del paseo muestra que el único lugar está ocupado.

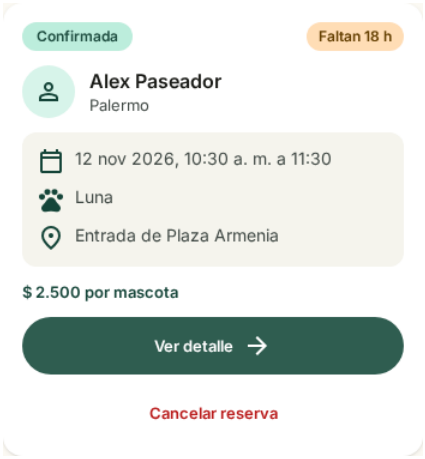



Figura 7. La reserva confirmada aparece en Próximas en la cuenta del dueño.




Figura 8. La reserva cancelada queda en Historial.

# 9. Finalización y reseña de un paseo

Después de volver a reservar, el paseador inició y finalizó el paseo. En el historial del dueño seleccionamos Calificar paseo, enviamos cinco estrellas y verificamos la calificación en el perfil del paseador.

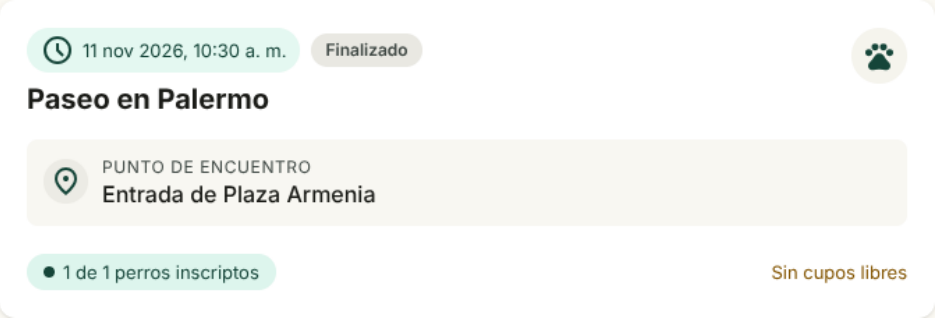



Figura 9. La pantalla del paseador muestra el paseo como finalizado.




Figura 10. El perfil muestra un promedio de cinco estrellas y una reseña.




Figura 11. La reseña publicada en una pantalla de 390 px.

Las cuentas y las mascotas son ficticias. El punto de encuentro se usó como ejemplo.

# 10. Instalación y recorrido de la demo

## Base de datos y configuración

Crear una base de datos en PostgreSQL 14 o superior. El backend necesita Java 21 y los datos de conexión a la base. Flyway crea el esquema y carga los barrios al iniciar.

```
DB_URL=jdbc:postgresql://localhost:5432/paseadores
DB_USER, DB_PASSWORD y JWT_SECRET: usar los valores locales.
```

Definir estas variables en la terminal o en un archivo backend/.env no versionado. El Dockerfile empaqueta el frontend y el backend juntos. PostgreSQL se ejecuta por separado. La guía del repositorio cubre ambas formas de ejecutar la aplicación.

## Terminales de backend y frontend

Desde la raíz del repositorio, abrir dos terminales. En la primera, iniciar el backend con el wrapper de Maven. En la segunda, instalar las dependencias del frontend con Node.js 24 y pnpm 11.25.0, y luego iniciar Vite.

```
cd backend && bash mvnw spring-boot:run
```

```
cd frontend && pnpm install --frozen-lockfile && pnpm dev
```

En Windows se puede usar mvnw.cmd. Abrir http://localhost:5173. Vite redirige /api a Spring Boot en http://localhost:8080. Swagger está disponible en /swagger-ui/index.html del backend.

## Pasos de la demo

1. Registrarse como paseador, agregar Palermo a la zona de servicio y publicar un paseo.
2. Registrarse como dueño, cargar una mascota, buscar por barrio y fecha, y reservar.
3. Cancelar desde Próximas mientras falten más de dos horas y volver a reservar.
4. Iniciar el paseo y revisar En curso. Finalizarlo y seleccionar Calificar paseo desde Historial.

# 11. Equipo y Alcance

## Cómo trabajamos

Planificamos el trabajo en ClickUp y colaboramos en GitHub. Juan Cruz trabajó en el análisis y en la integración de los módulos. Gabriel trabajó en el esquema y los scripts de la base de datos, y Gastón organizó la estructura del proyecto. Todos aportamos funcionalidades, pruebas y correcciones.

Trabajamos en el flujo del dueño, las reservas y la agenda del paseador. Usamos para el diseño, herramientas como Stitch para las pantallas de búsqueda, detalle y reserva del dueño. Los paseadores usan su propio perfil y agenda.

## Antes de la entrega

| Pendiente | Qué falta verificar |
|---|---|
| Revisión del equipo | Revisar el informe en conjunto y acordar el recorrido para la entrega. |
| Despliegue público | Desplegar la aplicación con PostgreSQL persistente y verificar el inicio de sesión y la reserva desde la URL pública. |
| Video de la demo | Grabar ambos roles con datos de ejemplo. Verificar el enlace y agregarlo al README y al informe. |

# 12. Conclusiones

El desarrollo de este MVP representó un desafío pero al mismo tiempo una excelente oportunidad para integrar y poner en práctica los conocimientos y herramientas adquiridos durante la cursada. A través de este proyecto, pudimos comprender cómo la tecnología puede transformar procesos cotidianos que antes se gestionaban de manera desorganizada. Nuestro objetivo es ofrecer una solución que permitiera centralizar los datos y convertirlos en información útil, accesible y relevante para los usuarios, facilitando la organización y gestión de los paseos.

Además, logramos implementar y verificar funcionalidades esenciales, como la actualización de cupos y estados entre diferentes cuentas, la gestión de reservas y la posibilidad de dejar reseñas una vez finalizado un paseo.

La presentación de un MVP funcional representa un logro importante como equipo, y también constituye el punto de partida de un proceso de mejora continua. A partir de esta primera versión, podremos incorporar nuevas funcionalidades de manera iterativa, tomando como referencia las necesidades y experiencias de los usuarios.

# Anexo: API y referencias

La API usa JSON y un token Bearer para las operaciones protegidas. Swagger está disponible en /swagger-ui/index.html. La tabla muestra las rutas principales de la demo.

| Método y ruta | Operación |
|---|---|
| POST /api/auth/register<br>POST /api/auth/login | Crear una cuenta o iniciar sesión. |
| GET /api/zones<br>GET / PUT /api/walkers/me | Consultar el catálogo o gestionar la cobertura de servicio propia. |
| GET / POST /api/walks<br>GET /api/walks/mine | Buscar o publicar paseos y consultar la agenda del paseador. |
| POST /api/walks/{id}/join/{petId}<br>GET /api/walks/bookings/mine | Reservar para una mascota propia. Listar Próximas (UPCOMING), En curso (IN_PROGRESS) o Historial (HISTORY). |
| DELETE /api/walks/{id}/join/{petId} | Cancelar una reserva propia. |
| PATCH /api/walks/{id}/status<br>POST /api/walks/{id}/reviews | Cambiar el estado de un paseo o reseñar un paseo finalizado como dueño participante. |

## Referencias

Repositorio del equipo: https://github.com/loveragabriel/trabajo-practico-integrador

Propuesta, arquitectura y alcance

Controlador de paseos y reservas

Guía de instalación local

Guía de identidad visual de la Universidad Tecnológica Nacional

Atribución de datos de OpenStreetMap

# Anexo: capturas adicionales

Los paseadores pueden ver el estado actual y las mascotas reservadas en un paseo. Cuando el paseador lo inicia, el dueño ve la reserva en En curso.

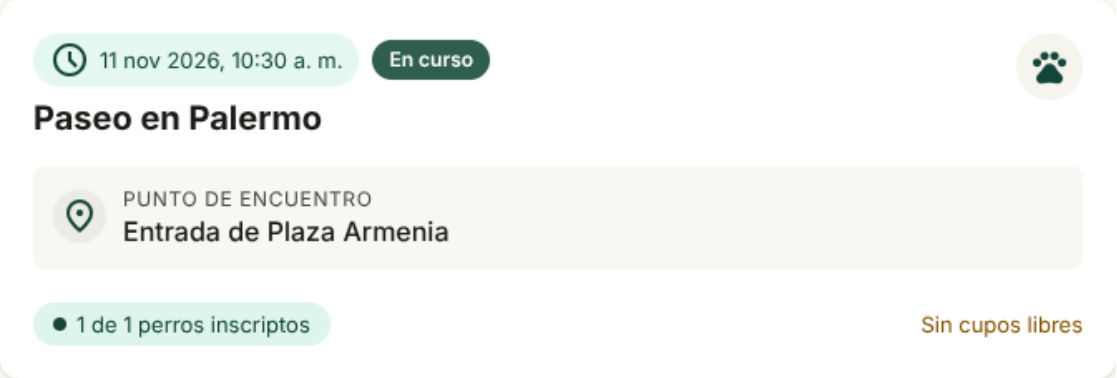



Figura 12. El paseo está en curso en la vista del paseador.

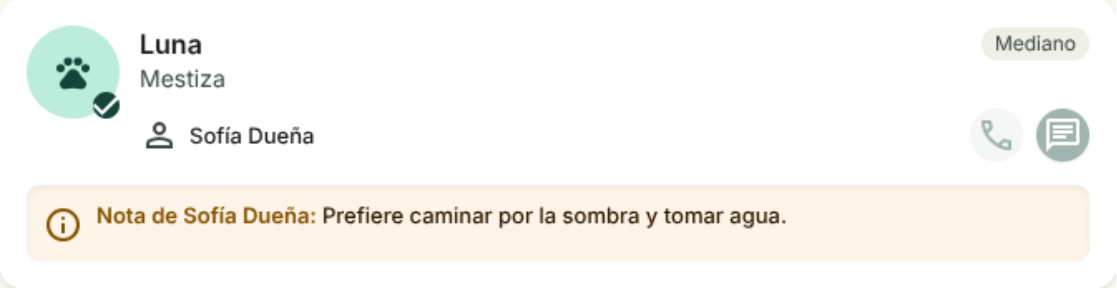



Figura 13. Una mascota reservada y las notas que ve el paseador.

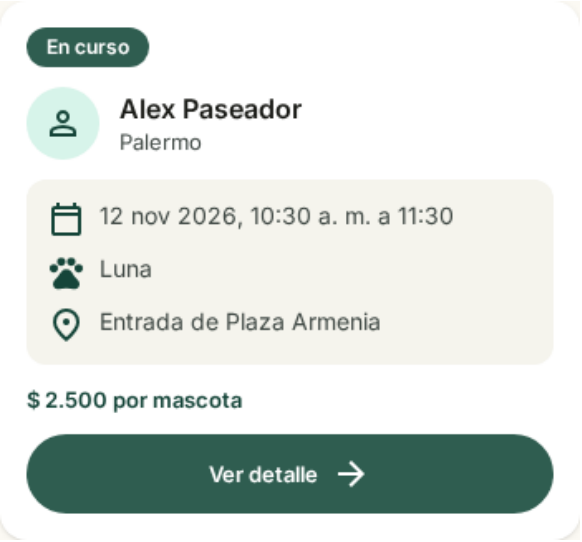



Figura 14. El dueño ve la reserva en En curso.